

Glosario - Semana 05

Composición de Componentes y Context API

A

API (Application Programming Interface)

Conjunto de funciones y procedimientos que permiten la comunicación entre componentes de software. En React, Context API es una interfaz para compartir datos entre componentes.

C

Children (Prop)

Propiedad especial en React que contiene los elementos JSX anidados dentro de un componente. Permite la composición de componentes.

```
interface CardProps {  
  children: ReactNode; // Acepta cualquier contenido válido de React  
}
```

Composition (Composición)

Patrón de diseño donde componentes complejos se construyen combinando componentes más simples. React favorece composición sobre herencia.

```
// Composición: Card contiene Header y Body  
<Card>  
  <Card.Header>Título</Card.Header>  
  <Card.Body>Contenido</Card.Body>  
</Card>
```

Compound Components

Patrón donde un componente padre y sus hijos trabajan juntos para lograr una funcionalidad, compartiendo estado implícitamente a través de Context.

```
// Compound Component: Tabs  
<Tabs>  
  <Tabs.List>  
    <Tabs.Tab>Tab 1</Tabs.Tab>  
  </Tabs.List>  
  <Tabs.Panel>Contenido 1</Tabs.Panel>  
</Tabs>
```

Consumer

Componente que lee valores de un Context. En la API moderna, se usa el hook `useContext` en lugar del componente Consumer.

Context

Mecanismo de React para pasar datos a través del árbol de componentes sin necesidad de pasar props manualmente en cada nivel (evita prop drilling).

createContext

Función de React que crea un objeto Context con Provider y Consumer.

```
const ThemeContext = createContext<Theme | undefined>(undefined);
```

D

Discriminated Union

Patrón de TypeScript donde cada tipo en una union tiene una propiedad literal que lo identifica únicamente.

```
type Action = { type: 'INCREMENT' } | { type: 'SET'; payload: number };
```

Dispatch

Función que envía acciones al reducer para actualizar el estado.

```
const [state, dispatch] = useReducer(reducer, initialState);  
dispatch({ type: 'INCREMENT' });
```

H

HOC (Higher-Order Component)

Función que recibe un componente y retorna un nuevo componente con funcionalidad adicional. Patrón menos usado desde la introducción de hooks.

```
const withTheme = (Component) => (props) => {  
  const theme = useTheme();  
  return <Component {...props} theme={theme} />;  
};
```

I

Implicit State Sharing

Compartir estado entre componentes sin pasarlo explícitamente via props, típicamente usando Context interno.

L

localStorage

API del navegador para almacenar datos persistentes en el cliente. Útil para guardar preferencias de usuario.

```
localStorage.setItem('theme', 'dark');  
const theme = localStorage.getItem('theme');
```

P

Prop Drilling

Anti-patrón donde props se pasan a través de múltiples niveles de componentes que no los necesitan, solo para llegar a componentes más profundos.

```
// ❌ Prop Drilling  
<App user={user}>  
  <Layout user={user}>  
    <Sidebar user={user}>  
      <UserProfile user={user} /> { /* Único que lo usa */ }  
    </Sidebar>  
  </Layout>  
</App>
```

```
    </Sidebar>
  </Layout>
</App>
```

Provider

Componente que hace disponible un valor de Context a todos sus descendientes.

```
<ThemeContext.Provider value={theme}>{children}</ThemeContext.Provider>
```

PropsWithChildren

Tipo de TypeScript que agrega `children: ReactNode` a un tipo de props.

```
type ButtonProps = PropsWithChildren<{
  onClick: () => void;
}>;
```

R

ReactElement

Tipo que representa un elemento React creado con JSX. Más específico que `ReactNode`.

```
interface SlotProps {
  header: ReactElement; // Solo elementos, no strings
}
```

ReactNode

Tipo que representa cualquier cosa que React puede renderizar: elementos, strings, números, null, undefined, arrays, fragments.

```
interface CardProps {
  children: ReactNode; // Acepta cualquier contenido
}
```

Reducer

Función pura que calcula el nuevo estado basándose en el estado actual y una acción.

```
const reducer = (state: State, action: Action): State => {
  switch (action.type) {
    case 'INCREMENT':
      return { count: state.count + 1 };
    default:
      return state;
  }
};
```

Render Props

Patrón donde un componente recibe una función como prop que retorna elementos React. Permite compartir lógica entre componentes.

```
<Mouse render={({mouse}) => <Cat position={mouse} />} />
```

S

Slots Pattern

Patrón de composición donde un componente define "ranuras" (slots) para contenido específico, permitiendo personalización flexible.

```
interface LayoutProps {  
  header: ReactNode;  
  sidebar: ReactNode;  
  content: ReactNode;  
}
```

State Colocation

Principio de mantener el estado lo más cerca posible de donde se usa. Solo elevar a Context cuando múltiples componentes lo necesitan.

U

useContext

Hook de React que permite leer valores de un Context.

```
const theme = useContext(ThemeContext);
```

useReducer

Hook de React para manejar estado complejo con un reducer. Alternativa a useState para lógica de estado más elaborada.

```
const [state, dispatch] = useReducer(reducer, initialState);
```

V

Value Prop

Propiedad del Provider que especifica el valor que estará disponible para todos los consumidores del Context.

```
<Context.Provider value={{ user, login, logout }}>
```

Referencias Rápidas

Término	Uso Principal
children	Composición básica
createContext	Crear contexto
useContext	Consumir contexto
useReducer	Estado complejo
Provider	Proveer valores

ReactNode	Tipo para children
Compound	Componentes relacionados
Slots	Múltiples áreas de contenido

Ejemplo Integrado

```
// Compound Component con Context interno
const TabsContext = createContext<TabsContextValue | undefined>(undefined);

const Tabs = Object.assign(
  ({ children, defaultTab }: TabsProps) => {
    const [activeTab, setActiveTab] = useState(defaultTab);
    return (
      <TabsContext.Provider value={{ activeTab, setActiveTab }}>
        <div className="tabs">{children}</div>
      </TabsContext.Provider>
    );
  },
  {
    List: TabsList, // Slot para lista de tabs
    Tab: Tab, // Tab individual
    Panel: TabPanel, // Panel de contenido
  },
);

// Uso
<Tabs defaultTab="home">
  <Tabs.List>
    <Tabs.Tab id="home">Inicio</Tabs.Tab>
    <Tabs.Tab id="settings">Config</Tabs.Tab>
  </Tabs.List>
  <Tabs.Panel id="home">Contenido inicio</Tabs.Panel>
  <Tabs.Panel id="settings">Contenido config</Tabs.Panel>
</Tabs>;
```